

PYTHON BASICS

Python Installation Command

sudo dnf install python3-pip -y

```
[ec2-user@ip-172-31-27-247 ~]$ sudo dnf install python3-pip -y
Last metadata expiration check: 0:03:15 ago on Fri Oct  2 19:45:09 2026.
Dependencies resolved.
=====
Package                Arch      Version                               Repository      Size
=====
Installing:
python3-pip            noarch    21.3.1-2.amzn2023.0.21              amazonlinux     1.8 M
Installing weak dependencies:
libxcrypt-compat       x86_64    4.4.33-7.amzn2023                   amazonlinux     92 k

Transaction Summary
=====
Install 2 Packages

Total download size: 1.9 M
Installed size: 11 M
Downloading Packages:
(1/2): libxcrypt-compat-4.4.33-7.amzn2023.x86_6 3.3 MB/s | 92 kB      00:00
(2/2): python3-pip-21.3.1-2.amzn2023.0.21.noarc 39 MB/s | 1.8 MB     00:00
-----
Total                                          19 MB/s | 1.9 MB     00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      :                                1/1
  Installing     : libxcrypt-compat-4.4.33-7.amzn2023.x86_64 1/2
  Installing     : python3-pip-21.3.1-2.amzn2023.0.21.noarch 2/2
  Running scriptlet: python3-pip-21.3.1-2.amzn2023.0.21.noarch 2/2
  Verifying      : libxcrypt-compat-4.4.33-7.amzn2023.x86_64 1/2
  Verifying      : python3-pip-21.3.1-2.amzn2023.0.21.noarch 2/2

Installed:
  libxcrypt-compat-4.4.33-7.amzn2023.x86_64
  python3-pip-21.3.1-2.amzn2023.0.21.noarch

Complete!
```

Python Version

```
[ec2-user@ip-172-31-27-247 ~]$ which python3
/usr/bin/python3
[ec2-user@ip-172-31-27-247 ~]$ python3 --version
Python 3.9.25
```

Print the Values

```
>>> print ("WELCOME TO PYTHON PROGRAMMING LANGUAGE")
WELCOME TO PYTHON PROGRAMMING LANGUAGE
>>> PRINT ("Welcome to AWS DATA ENGINEER")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'PRINT' is not defined
>>> print ("Welcome to AWS DATA ENGINEER")
Welcome to AWS DATA ENGINEER
>>> print ("Learning Python Programming Language")
Learning Python Programming Language
>>> █
```

Data Engineer Procedure

```
>>> print("Pipeline Started")
Pipeline Started
>>> print("Reading customer data")
Reading customer data
>>> print("Processing completed")
Processing completed
>>> █
```

3) VARIABLES

A variable is a name used to store a value.

Name = "Arun"

Name – Variables – Store the Value

Arun – Values – Actual data stored

= - Assignment Operator – Assigns values

```
>>> name = "Arun"
>>> age = 30
>>> salary = 50000
>>> print(name)
Arun
>>> print(age)
30
>>> print(salary)
50000
```

```
source_file = "customer data.csv"
bucket_name = "company-Raw Data"
record_count = 10000
```

```
>>> source_file = "Customer data.csv"
>>>
>>> bucket_name = "Company Raw Data"
>>>
>>> record_count = 10000
>>>
```

Print the Values

```
>>> print(source_file)
Customer data.csv
>>>
>>>
>>> print(bucket_name)
Company Raw Data
>>>
>>>
>>> print(record_count)
10000
```

Python is case sensitive

```
>>> name = "Arun"
>>> Name = "Priya"
>>>
>>>
>>>
>>> print (name)
Arun
>>>
>>> print (Name)
Priya
```

4) BASIC DATA TYPE

NAME = NIRMAL – STRING

ID = 101 – INT

Account Balance = FLOAT

True/False = Boolean

```
>>> customer_name = "Nirmal"
>>> customer_id = 101
>>> account_balance = 125000.50
>>> is_active = True
>>>
>>>
>>>
>>> print(type(customer_name))
<class 'str'>
>>>
>>>
>>> print(type(customer_id))
<class 'int'>
>>>
>>> print(type(account_balance))
<class 'float'>
>>>
>>> print(type(is_active))
<class 'bool'>
>>> █
```

5) TYPE CONVENTION

A value can look like a number but still be stored as a string.

Convert string to integer:

```
>>> customer_id = "101"
>>>
>>> print(type(customer_id))
<class 'str'>
>>>
>>>
>>>
>>> customer_id = int(customer_id)
>>>
>>> print (customer_id)
101
>>>
>>> print (type(customer_id))
<class 'int'>
>>>
```

Convert string to float

```
>>> amount = 50000.75
>>> amount = float(amount)
>>>
>>> print(amount)
50000.75
>>>
>>> print(type(amount))
<class 'float'>
>>>
```

Conversions

```
>>> quantity = "50"
>>>
>>> price = "7500"
>>>
>>> total = int(quantity) * int(price)
>>> print(total)
375000
```

6) OPERATORS

A = 70

B = 30

```
>>> a = 70
>>>
>>> b = 30
>>>
>>> print(a + b)
100
>>> print(a - b)
40
>>> print(a * b)
2100
>>> print(a / b)
2.3333333333333335
```

Data pipeline calculation

```
>>> total_records = 5000
>>>
>>> failed_records = 750
>>>
>>> successfull_records = total_records - failed_records
>>>
>>> success_percentage = (successfull_records / total_records) * 100
>>>
>>> print(successfull_records)
4250
>>>
>>> print(success_percentage)
85.0
>>>
```

7) STRINGS

A string is a sequence of characters enclosed in quotes and used to represent text.

- Strings are used for names, file names, paths, S3 bucket names, JSON values and SQL text.

first_name - contains the user's first name.

" " - is a string containing a single space.

last_name - contains the user's last name.

The **+** operator concatenates (joins) strings together.

```
>>> first_name = "John"
>>>
>>>
>>> last_name = "Hendry"
>>>
>>> full_name = first_name + " " + last_name
>>>
>>> print(full_name)
John Hendry
```

+ joins (concatenates) the two strings

```
>>> first_name = "John"
>>>
>>> last_name = "Hendry"
>>>
>>> full_name = first_name + last_name
>>>
>>> print(full_name)
JohnHendry
>>>
```

Strings Methods

Print the Upper, Lower and Title of the strings

```
>>> name = "aws data engineering"
>>>
>>>
>>> print(name.upper())
AWS DATA ENGINEERING
>>>
>>> print(name.lower())
aws data engineering
>>>
>>>
>>> print(name.title())
Aws Data Engineering
>>>
```

strip() - Remove unwanted spaces

```
>>> customer_name = "   John   "
>>>
>>> print(customer_name.strip())
John
```

replace() – Replace text

```
>>> file_name = "customer_data.csv"
>>>
>>> new_name = file_name.replace(".csv", ".json")
>>>
>>> print(new_name)
customer_data.json
>>>
```

startswith() – Check starting text

```
>>> file_name = "customer_data_2026.csv"
>>>
>>>
>>> print(file_name.startswith("cust"))
True
>>>
>>> print(file_name.startswith("data"))
False
>>>
>>> print(file_name.startswith("customer"))
True
>>>
>>> print(file_name.startswith("Customer"))
False
```

endswith() - Check ending text

```
>>> file_name = "customer_data_2026.csv"
>>>
>>> print(file_name.endswith(".csv"))
True
>>>
>>> print(file_name.endswith(".cust"))
False
>>>
>>> print(file_name.endswith("cust"))
False
>>>
>>> print(file_name.endswith(".CSV"))
False
```

8) USER INPUT – INPUT()

The input() function is used to receive input from the user through the keyboard during program execution.

```
>>> name = input("Enter your name: ")
Enter your name: John
>>>
>>> print("Hello", name)
Hello John
>>>
>>> age = int(input("Enter your age: "))
Enter your age: 10
>>> print("Next Year Age Will Be", age+1)
Next Year Age Will Be 11
>>>
```

8) HANDS ON PRACTISE

```
source_file = "customers.csv"
total_records = 15000
failed_records = 750
processing_time = 60 seconds
```

Output:

```
Successful Records: 14250
Success Percentage: 95.0
```

```

>>> source_file = "customer.csv"
>>>
>>> total_records = 15000
>>>
>>> failed_records = 750
>>>
>>> processing_time = 60
>>>
>>> successfull_records = total_records - failed_records
>>>
>>> print(source_file:", source_file)
File "<stdin>", line 1
    print(source_file:", source_file)
          ^
SyntaxError: invalid syntax
>>> print("source_file:", source_file)
source_file: customer.csv
>>> print("total_records:", total_records)
total_records: 15000
>>>
>>> print("failed_records:", failed_records)
failed_records: 750
>>>
>>> print("processing_time:", processing_time)
processing_time: 60
>>> print("successfull_records:", successfull_records)
successfull_records: 14250
>>>
>>>
>>>
>>>
>>> success_percentage = (successful_records / total_records) * 100
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'successful_records' is not defined
>>> success_percentage = (successfull_records / total_records) * 100
>>>
>>>
>>> print("Success Percentage:", success_percentage)
Success Percentage: 95.0
>>>

```

9) FINAL PRACTISE

File Name	orders.csv
Total Records	50000
Failed Records	1250
Processing Time	120.5 seconds
Environment	production

Write a Python program to display the data and calculate:

- Successful Records
- Failed Records
- Success Percentage
- Failure Percentage

```
>>> file_name = "order.csv"
>>>
>>> total_records = 50000
>>>
>>> failed_records = 5780
>>>
>>> processing_time = 150.78
>>>
>>> environment = "production"
>>>
>>> print("file_name:", file_name)
file_name: order.csv
>>>
>>> print("total_records:", total_records)
total_records: 50000
>>>
>>> print("failed_records:", failed_records)
failed_records: 5780
>>>
>>> print("processing_time", processing_time)
processing_time 150.78
>>>
>>> print("environment", environment)
environment production
>>>
>>> print("environment:", environment)
environment: production
>>>
>>> successfull_records = total_records - failed_records
>>>
>>> successfull_precentage = (successfull_records / total_records) * 100
>>>
>>> print("successfull_records:", successfull_records)
successfull_records: 44220
>>>
>>> print ("successfull_precentage:", successfull_precentage)
successfull_precentage: 88.44
>>>
>>>
>>> failed_precentage = (failed_records / total_records) * 100
>>>
>>> print ("failed_precentage:", failed_precentage)
failed_precentage: 11.559999999999999
>>>
>>> □
```